

Lab Assignments

PRACTICAL NO.01

Practice Lab Assignments

1. Program to Calculate Momentum

```
-----  
Enter mass in kg: 23  
Enter velocity in m/s: 45  
Momentum = 46575.0
```

```
=====
```

```
Enter a number: 45  
Square Root: 6.708203932499369  
|
```

2. Program Based on Number of Digits

```
2nd.py - D:/MIT AOE 1ST YR/Python/2nd.py (3.14.4)  
File Edit Format Run Options Window Help  
import math  
n = int(input("Enter a number: "))  
digits = len(str(abs(n)))  
  
if digits == 1:  
    print("Square:", n ** 2)  
elif digits == 2:  
    print("Square Root:", math.sqrt(n))  
elif digits == 3:  
    print("Cube Root:", n ** (1/3))  
else:  
    print("Number has more than three digits")
```

```
=====
```

```
Enter a number: 45  
Square Root: 6.708203932499369  
|
```

```
=====
```

```
Enter a number: 123  
Cube Root: 4.97318983326859
```

3. Data Transformation Using Functions

```
3rd.py - D:/MIT AOE 1ST YR/Python/3rd.py (3.14.4)
File Edit Format Run Options Window Help
from datetime import date

def calculate_age(year):
    return date.today().year - year

def rupees_to_dollars(salary):
    return salary / 83 # Approx conversion rate

birth_year = int(input("Enter birth year: "))
salary_rs = float(input("Enter salary in rupees: "))

print("Age:", calculate_age(birth_year))
print("Salary in dollars:", rupees_to_dollars(salary_rs))
|

-----
Enter birth year: 2007
Enter salary in rupees: 46000
Age: 19
Salary in dollars: 554.2168674698795
|
```

4. Reverse a Given Number

```
File Edit Format Run Options Window Help
num = int(input("Enter a number: "))
rev = 0

while num > 0:
    rev = rev * 10 + num % 10
    num //= 10

print("Reversed Number:", rev)
|

=====
Enter a number: 13
Reversed Number: 31
```

Lab Assignments

1. Student Result and Grade Calculation

```

File Edit Format Run Options Window Help
marks = []
for i in range(5):
    marks.append(int(input(f"Enter marks of subject {i+1}: ")))

if min(marks) < 40:
    print("Result: Fail")
else:
    total = sum(marks)
    percentage = total / 5

    print("Result: Pass")
    print("Percentage:", percentage)

    if percentage > 75:
        print("Grade: Distinction")
    elif percentage >= 60:
        print("Grade: First Division")
    elif percentage >= 50:
        print("Grade: Second Division")
    else:
        print("Grade: Third Division")

=====
Enter marks of subject 1: 38
Enter marks of subject 2: 56
Enter marks of subject 3: 58
Enter marks of subject 4: 98
Enter marks of subject 5: 67
Result: Fail
|

```

2. Fibonacci Series Using Recursive Function

```

def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

terms = int(input("Enter number of terms: "))
for i in range(terms):
    print(fibonacci(i), end=" ")

=====
Enter number of terms: 3
0 1 1

=====
Enter number of terms: 6
0 1 1 2 3 5

```

3. Pattern Printing Program

File	Edit	Format	Run	Options	Window	Help	=====
							Enter number of rows: 4
n = int(input("Enter number of rows: "))							*
							* *
for i in range(1, n+1):							* * *
print("* " * i)							* * * *

Self Study Assignment

Statistical Analysis on Employee Salary Data

File	Edit	Format	Run	Options	Window	Help

salaries = [35000, 42000, 50000, 28000, 60000]						
print("Average Salary:", sum(salaries)/len(salaries))						
print("Maximum Salary:", max(salaries))						
print("Minimum Salary:", min(salaries))						
print("Total Salary:", sum(salaries))						
=====						
Average Salary: 43000.0						
Maximum Salary: 60000						
Minimum Salary: 28000						
Total Salary: 215000						

PRACTICAL NO.02

Practice Lab Assignment

1. Perform All List Operations

```
File Edit Format Run Options Window Help
```

```
# Creating a list
```

```
lst = [10, 20, 30, 40]
```

```
# Append
```

```
lst.append(50)
```

```
# Insert
```

```
lst.insert(2, 25)
```

```
# Remove
```

```
lst.remove(20)
```

```
# Pop
```

```
lst.pop()
```

```
# Update
```

```
lst[0] = 5
```

```
# Length
```

```
print("Length:", len(lst))
```

```
# Sorting
```

```
lst.sort()
```

```
# Reverse
```

```
lst.reverse()
```

```
print("Final List:", lst)
```

```
=====
Length: 4
Final List: [40, 30, 25, 5]
```

2. Perform All Dictionary Operations

File Edit Format Run Options Window Help

```
# Creating dictionary
student = {
    "roll": 101,
    "name": "Amit",
    "marks": 85
}

# Add element
student["grade"] = "A"

# Update value
student["marks"] = 90

# Access value
print("Name:", student["name"])

# Remove element
student.pop("roll")

# Keys and Values
print("Keys:", student.keys())
print("Values:", student.values())

print("Final Dictionary:", student)

=====
Name: Amit
Keys: dict_keys(['name', 'marks', 'grade'])
Values: dict_values(['Amit', 90, 'A'])
Final Dictionary: {'name': 'Amit', 'marks': 90, 'grade': 'A'}
```

Lab Assignment

1. Linear Search – Find Position of a Number

```

File Edit Format Run Options Window Help
lst = [10, 25, 30, 45, 50]
key = int(input("Enter number to search: "))

found = False
lent=len(lst)
for i in range(lent):
    if lst[i] == key:
        print("Element found at position:", i + 1)
        found = True
        break

if not found:
    print("Element not found")

=====
Enter number to search: 35
Element not found

=====
Enter number to search: 30
Element found at position: 3

```

2. Cricket Team – Find Captain (Tallest Player)

```

team = {
    "Player1": 170,
    "Player2": 175,
    "Player3": 180,
    "Player4": 178,
    "Player5": 172,
    "Player6": 176,
    "Player7": 174,
    "Player8": 179,
    "Player9": 173,
    "Player10": 177,
    "Player11": 181
}

captain = max(team, key=team.get)
print("Captain of the Team:", captain)
print("Height:", team[captain])

=====
Captain of the Team: Player11
Height: 181

```

Self Study Assignment

1. Student Marks – Total and Percentage

13th.py - D:/MIT AOE 1ST YR/Python/13th.py (3.14.4)

File Edit Format Run Options Window Help

```
marks = [78, 85, 69, 74, 88]

total = sum(marks)
percentage = total / len(marks)

print("Total Marks:", total)
print("Percentage:", percentage)
```

=====

```
Total Marks: 394
Percentage: 78.8
|
```

2. Perform All Tuple Operations

14th.py - D:/MIT AOE 1ST YR/Python/14th.py (3.14.4)

File Edit Format Run Options Window Help

```
# Creating tuple
t = (10, 20, 30, 40, 50)

# Access
print("First Element:", t[0])

# Length
print("Length:", len(t))

# Count
print("Count of 20:", t.count(20))

# Index
print("Index of 30:", t.index(30))
|
# Concatenation
t2 = t + (60, 70)
print("After Concatenation:", t2)

# Repetition
print("Repetition:", t * 2)

# Slicing
print("Slicing:", t[1:4])
```

=====

```
First Element: 10
Length: 5
Count of 20: 1
Index of 30: 2
After Concatenation: (10, 20, 30, 40, 50, 60, 70)
Repetition: (10, 20, 30, 40, 50, 10, 20, 30, 40, 50)
Slicing: (20, 30, 40)
```


PRACTICAL NO.03

Practice Lab Assignment

1. Perform All NumPy Operations

File Edit Format Run Options Window Help

```
import numpy as np

# Create arrays
a = np.array([10, 20, 30, 40])
b = np.array([1, 2, 3, 4])

# Basic operations
print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)

# Shape and size
print("Shape:", a.shape)
print("Size:", a.size)
print("Data Type:", a.dtype)

# Reshape
c = a.reshape(2, 2)
print("Reshaped Array:\n", c)

# Zeros and Ones
print("Zeros:\n", np.zeros((2,2)))
print("Ones:\n", np.ones((3,3)))
```

```
-----
Addition: [11 22 33 44]
Subtraction: [ 9 18 27 36]
Multiplication: [ 10 40 90 160]
Division: [10. 10. 10. 10.]
Shape: (4,)
Size: 4
Data Type: int64
Reshaped Array:
[[10 20]
 [30 40]]
Zeros:
[[0. 0.]
 [0. 0.]]
Ones:
[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]
```

Lab Assignment

Dataset: Student Marks (Real-Life Example)

- 1. Perform All Matrix Operations**
- 2. Horizontal and Vertical Stacking**
- 3. Custom Sequence Generation**
- 4. Arithmetic, Statistical & Mathematical Operations**
- 5. Bitwise Operators**
- 6. Copying and Viewing Arrays**
- 7. Data Stacking, Searching, Sorting, Counting**
- 8. Broadcasting**

```
import numpy as np

data = np.array([
    [78, 85, 69],
    [88, 76, 90],
    [92, 81, 77]
])

print("Dataset:\n", data)

print("Transpose:\n", data.T)
print("Matrix Addition:\n", data + data)
print("Matrix Subtraction:\n", data - 10)
print("Matrix Multiplication:\n", np.dot(data, data.T))

a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

print("Horizontal Stack:\n", np.hstack((a, b)))
print("Vertical Stack:\n", np.vstack((a, b)))

print("Arange:", np.arange(1, 11))
print("Linspace:", np.linspace(1, 10, 5))
print("Logspace:", np.logspace(1, 3, 4))

print("Sum:", np.sum(data))
print("Mean:", np.mean(data))
print("Max:", np.max(data))
print("Min:", np.min(data))
print("Standard Deviation:", np.std(data))

print("Square Root:\n", np.sqrt(data))
print("Power:\n", np.power(data, 2))

x = np.array([5, 3])
y = np.array([2, 1])

print("Bitwise AND:", np.bitwise_and(x, y))
print("Bitwise OR:", np.bitwise_or(x, y))
print("Bitwise XOR:", np.bitwise_xor(x, y))

original = np.array([10, 20, 30])
view_arr = original.view()
copy_arr = original.copy()

original[0] = 100

print("Original:", original)
print("View:", view_arr)
print("Copy:", copy_arr)

arr = np.array([10, 20, 10, 30, 40])

# Searching
print("Index of 30:", np.where(arr == 30))

# Sorting
print("Sorted Array:", np.sort(arr))

# Counting
print("Count of 10:", np.count_nonzero(arr == 10))

# Unique values
print("Unique Elements:", np.unique(arr))

matrix = np.array([[1, 2, 3], [4, 5, 6]])
scalar = 10

print("Broadcasting Result:\n", matrix + scalar)
1
|
```

```

Dataset:
[[78 85 69]
 [88 76 90]
 [92 81 77]]
Transpose:
[[78 88 92]
 [85 76 81]
 [69 90 77]]
Matrix Addition:
[[156 170 138]
 [176 152 180]
 [184 162 154]]
Matrix Subtraction:
[[68 75 59]
 [78 66 80]
 [82 71 67]]
Matrix Multiplication:
[[18070 19534 19374]
 [19534 21620 21182]
 [19374 21182 20954]]
Horizontal Stack:
[[1 2 5 6]
 [3 4 7 8]]
Vertical Stack:
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
Arange: [ 1  2  3  4  5  6  7  8  9 10]
Linspace: [ 1.    3.25  5.5   7.75 10.   ]
Logspace: [ 10.    46.41588834 215.443469 1000.    ]
Sum: 736
Mean: 81.77777777777777
Max: 92
Min: 69
Standard Deviation: 7.114582486036498
Square Root:
[[8.83176087 9.21954446 8.30662386]
Square Root:
[[8.83176087 9.21954446 8.30662386]
 [9.38083152 8.71779789 9.48683298]
 [9.59166305 9.    8.77496439]]
Power:
[[6084 7225 4761]
 [7744 5776 8100]
 [8464 6561 5929]]
Bitwise AND: [0 1]
Bitwise OR: [7 3]
Bitwise XOR: [7 2]
Original: [100 20 30]
View: [100 20 30]
Copy: [10 20 30]
Index of 30: (array([3]),)
Sorted Array: [10 10 20 30 40]
Count of 10: 2
Unique Elements: [10 20 30 40]
Broadcasting Result:
[[11 12 13]
 [14 15 16]]
>>>

```

Self Study Assignment

Image as Array and Image Manipulation (Real-Life Application)

```
import numpy as np
import cv2
import matplotlib.pyplot as plt

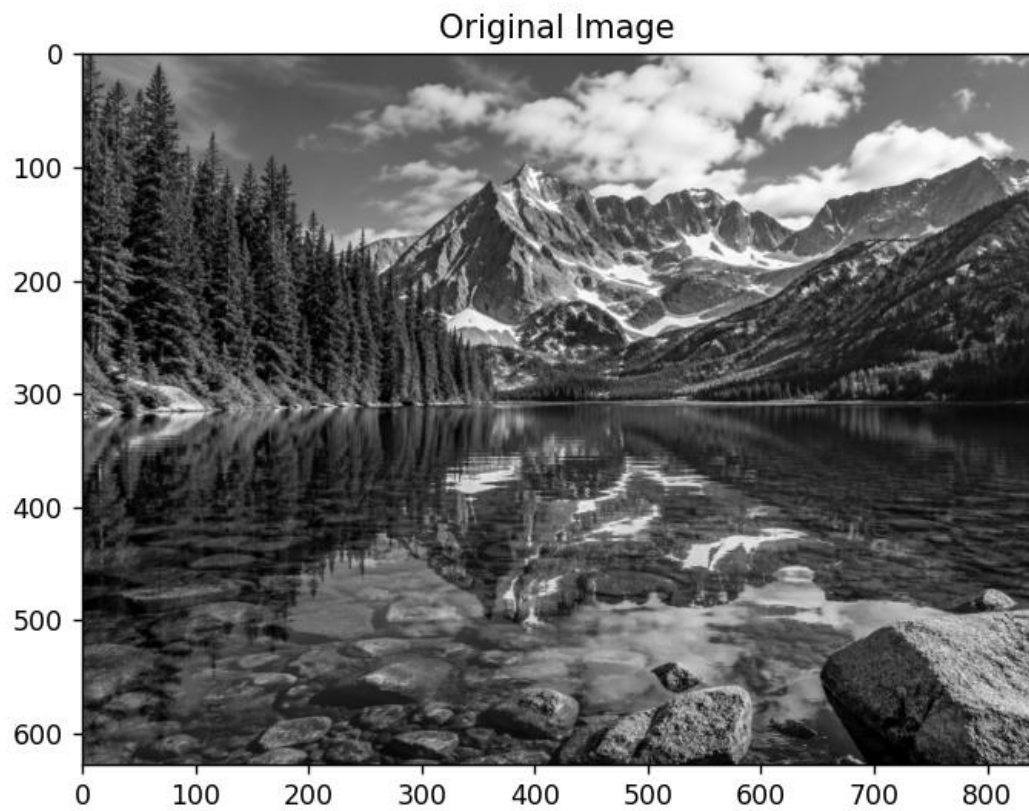
# Read image
img = cv2.imread('image.jpg', 0) # grayscale

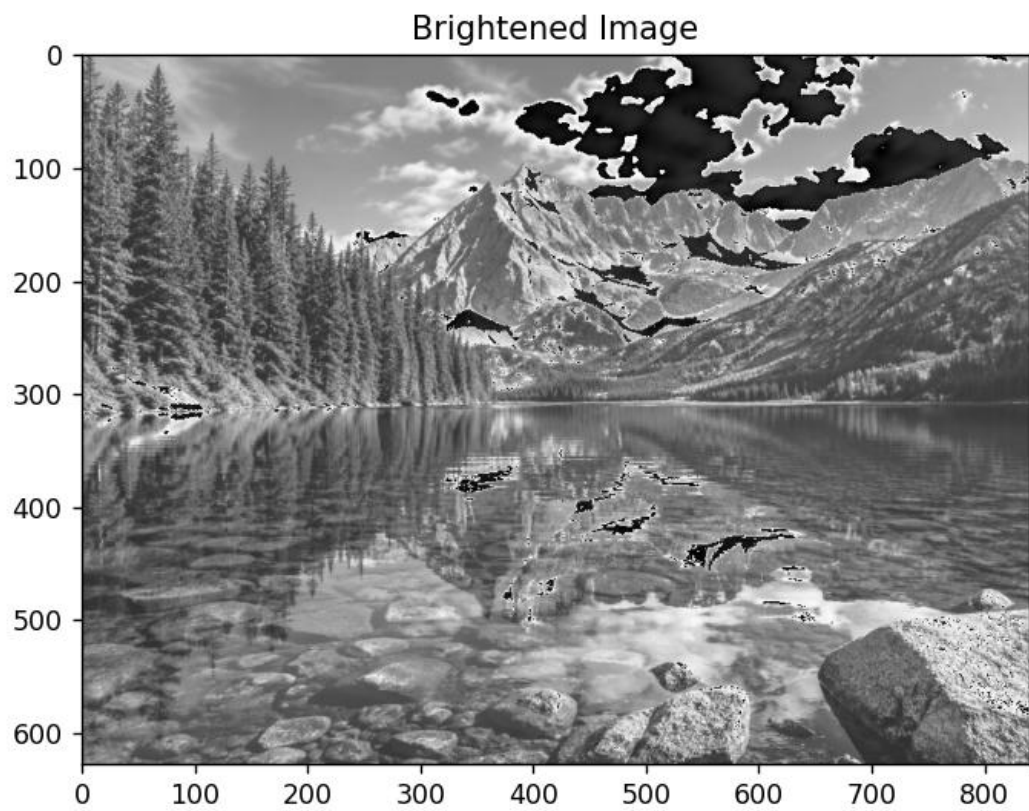
# Display image
plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.show()

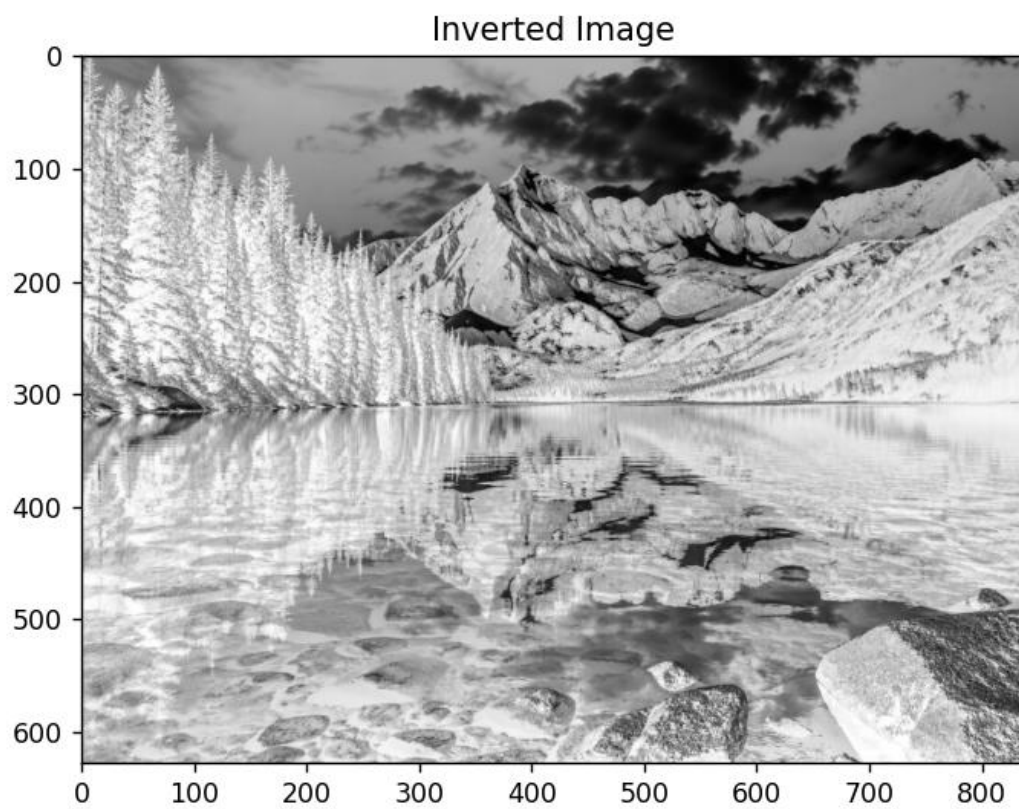
# Image operations
print("Image Shape:", img.shape)

# Brightness increase
bright = img + 50
plt.imshow(bright, cmap='gray')
plt.title("Brightened Image")
plt.show()

# Image inversion
invert = 255 - img
plt.imshow(invert, cmap='gray')
plt.title("Inverted Image")
plt.show()
```







PRACTICAL NO.04

1. Perform All Pandas Operations

```
import pandas as pd

# Create DataFrame
data = {
    "Name": ["Amit", "Neha", "Ravi", "Priya"],
    "Age": [21, 22, 20, 23],
    "Marks": [78, 85, 69, 90]
}

df = pd.DataFrame(data)

# Display
print(df)

# Basic operations
print("Head:\n", df.head())
print("Tail:\n", df.tail())
print("Shape:", df.shape)
print("Columns:", df.columns)
print("Data Types:\n", df.dtypes)

# Selection
print(df["Marks"])
print(df.loc[1])
print(df.iloc[2])

# Sorting
print(df.sort_values("Marks"))

# Filtering
print(df[df["Marks"] > 80])

# Add column
df["Result"] = ["Pass", "Pass", "Pass", "Pass"]

# Drop column
df.drop("Age", axis=1, inplace=True)

print("Updated DataFrame:\n", df)
```

```

      Name  Age  Marks
0    Amit   21    78
1    Neha   22    85
2    Ravi   20    69
3    Priya  23    90
Head:
      Name  Age  Marks
0    Amit   21    78
1    Neha   22    85
2    Ravi   20    69
3    Priya  23    90
Tail:
      Name  Age  Marks
0    Amit   21    78
1    Neha   22    85
2    Ravi   20    69
3    Priya  23    90
Shape: (4, 3)
Columns: Index(['Name', 'Age', 'Marks'], dtype='str')
Data Types:
      Name      str
      Age    int64
      Marks    int64
dtype: object
0      78
1      85
2      69
3      90
Name: Marks, dtype: int64
Name      Neha
Age        22
Marks      85
Name: 1, dtype: object
Name      Ravi
Age        20
Marks      69
Name: 2, dtype: object
      Name  Age  Marks
2    Ravi   20    69
0    Amit   21    78
1    Neha   22    85
3    Priya  23    90
      Name  Age  Marks
1    Neha   22    85
3    Priya  23    90
Updated DataFrame:
      Name  Marks Result
0    Amit     78    Pass
1    Neha     85    Pass
2    Ravi     69    Pass
3    Priya    90    Pass
|

```

Lab Assignment

Dataset: Sales Dataset (Real-Life Example)


```

df["Sales"] = df["Quantity Ordered"] * df["Price Each"]

df["Order Date"] = pd.to_datetime(df["Order Date"])
df["Month"] = df["Order Date"].dt.month
df["City"] = df["Purchase Address"].apply(lambda x: x.split(",")[1])

monthly_sales = df.groupby("Month")["Sales"].sum()
print(monthly_sales)

print("Best Month:", monthly_sales.idxmax())
print("Highest Sales:", monthly_sales.max())

product_qty = df.groupby("Product")["Quantity Ordered"].sum()
print(product_qty.sort_values(ascending=False).head(1))

city_sales = df.groupby("City")["Sales"].sum()
print(city_sales.sort_values(ascending=False))

df_dup = df[df["Order ID"].duplicated(keep=False)]
df_dup["Grouped"] = df_dup.groupby("Order ID")["Product"].transform(lambda x: ", ".join(x))
print(df_dup["Grouped"].value_counts().head())

print("Average Sales:", df["Sales"].mean())

print("Max Sale:", df["Sales"].max())

print("Min Sale:", df["Sales"].min())

print("Total Revenue:", df["Sales"].sum())

print(df.groupby("Product")["Price Each"].max().sort_values(ascending=False).head(1))

print(df["City"].value_counts())

print(df["Sales"].median())
print(df["Sales"].std())
print(df.groupby("Month").size())
print(df["Product"].nunique())
print(df.groupby("City")["Sales"].mean())
print(df.groupby("Product")["Sales"].sum().nlargest(5))
print(df.groupby("Product")["Sales"].sum().nsmallest(5))
print(df["Sales"].cumsum())
print(df.describe())
print(df.corr(numeric_only=True))

```

```

Month
1      90000
2      25000
3      54000
4      25000
5       6000
6       4800
Name: Sales, dtype: int64
Best Month: 1
Highest Sales: 90000
Product
Mouse      6
Name: Quantity Ordered, dtype: int64
City
Maharashtra    144800
Delhi           54000
Karnataka       6000
Name: Sales, dtype: int64
Series([], Name: count, dtype: int64)
Average Sales: 34133.333333333336
Max Sale: 90000
Min Sale: 4800
Total Revenue: 204800
Product
Laptop      45000
Name: Price Each, dtype: int64
City
Maharashtra    4
Delhi           1
Karnataka       1
Name: count, dtype: int64
25000.0
32663.537265070765
Month
1      1
2      1
3      1
4      1
5      1
6      1
dtype: int64
6

```

```

City
Delhi          54000.0
Karnataka      6000.0
Maharashtra    36200.0
Name: Sales, dtype: float64
Product
Laptop         90000
Tablet         54000
Headphones     25000
Mobile         25000
Keyboard       6000
Name: Sales, dtype: int64
Product
Mouse          4800
Keyboard       6000
Headphones     25000
Mobile         25000
Tablet         54000
Name: Sales, dtype: int64
0      90000
1     115000
2     169000
3     194000
4     200000
5     204800
Name: Sales, dtype: int64

```

	Order ID	Quantity Ordered	...	Sales	Month
count	6.000000	6.000000	...	6.000000	6.000000
mean	1003.500000	3.500000	...	34133.333333	3.500000
min	1001.000000	1.000000	...	4800.000000	1.000000
25%	1002.250000	2.250000	...	10750.000000	2.250000
50%	1003.500000	3.500000	...	25000.000000	3.500000
75%	1004.750000	4.750000	...	46750.000000	4.750000
max	1006.000000	6.000000	...	90000.000000	6.000000
std	1.870829	1.870829	...	32663.537265	1.870829

```

[8 rows x 6 columns]

```

	Order ID	Quantity Ordered	Price Each	Sales	Month
Order ID	1.000000	0.885714	-0.943423	-0.837862	1.000000
Quantity Ordered	0.885714	1.000000	-0.797804	-0.562939	0.885714
Price Each	-0.943423	-0.797804	1.000000	0.898612	-0.943423
Sales	-0.837862	-0.562939	0.898612	1.000000	-0.837862
Month	1.000000	0.885714	-0.943423	-0.837862	1.000000

Self Study Assignment

Advanced Data Manipulation

1. Combining Data (Merge & Concatenate)

```
import pandas as pd
df1 = pd.DataFrame({"ID": [1,2], "Name": ["A", "B"]})
df2 = pd.DataFrame({"ID": [1,2], "Salary": [30000, 40000]})

merged = pd.merge(df1, df2, on="ID")
print(merged)

combined = pd.concat([df1, df1])
print(combined)

=====
   ID Name  Salary
0    1    A   30000
1    2    B   40000
   ID Name
0    1    A
1    2    B
0    1    A
1    2    B
|
```

2. Time Series Data Handling

```
df1.set_index("Order Date", inplace=True)

# Monthly sales trend
monthly_trend = df["Sales"].resample("M").sum()
print(monthly_trend)
```

```
   ID Name  Salary
0    1    A   30000
1    2    B   40000
   ID Name
0    1    A
1    2    B
0    1    A
1    2    B
```

PRACTICAL NO.05

Practice Lab Assignment

1. Install Matplotlib Library

pip install matplotlib

2. Draw Basic Graphs for Sales Dataset

Sample Sales Dataset

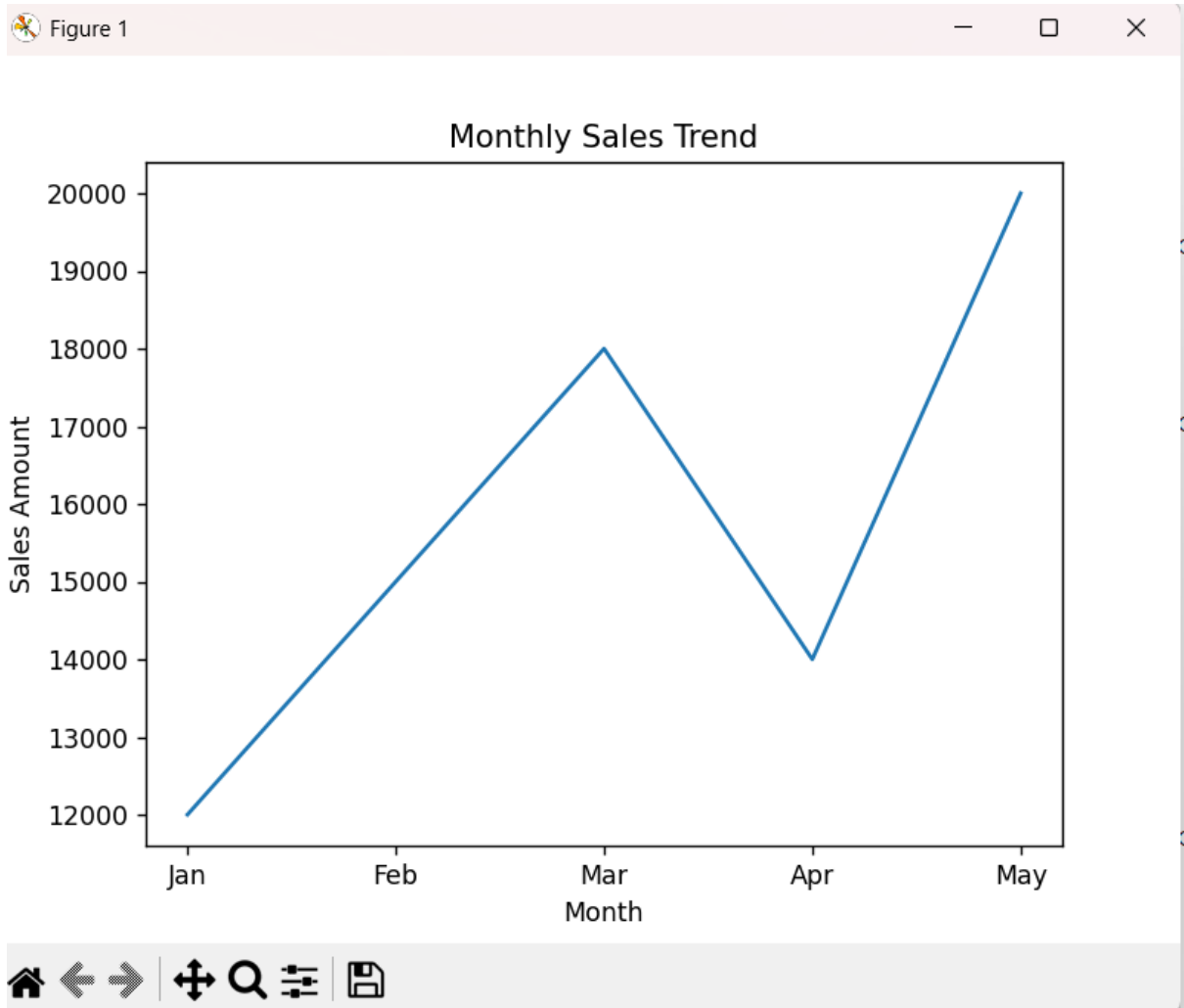

```
import matplotlib.pyplot as plt

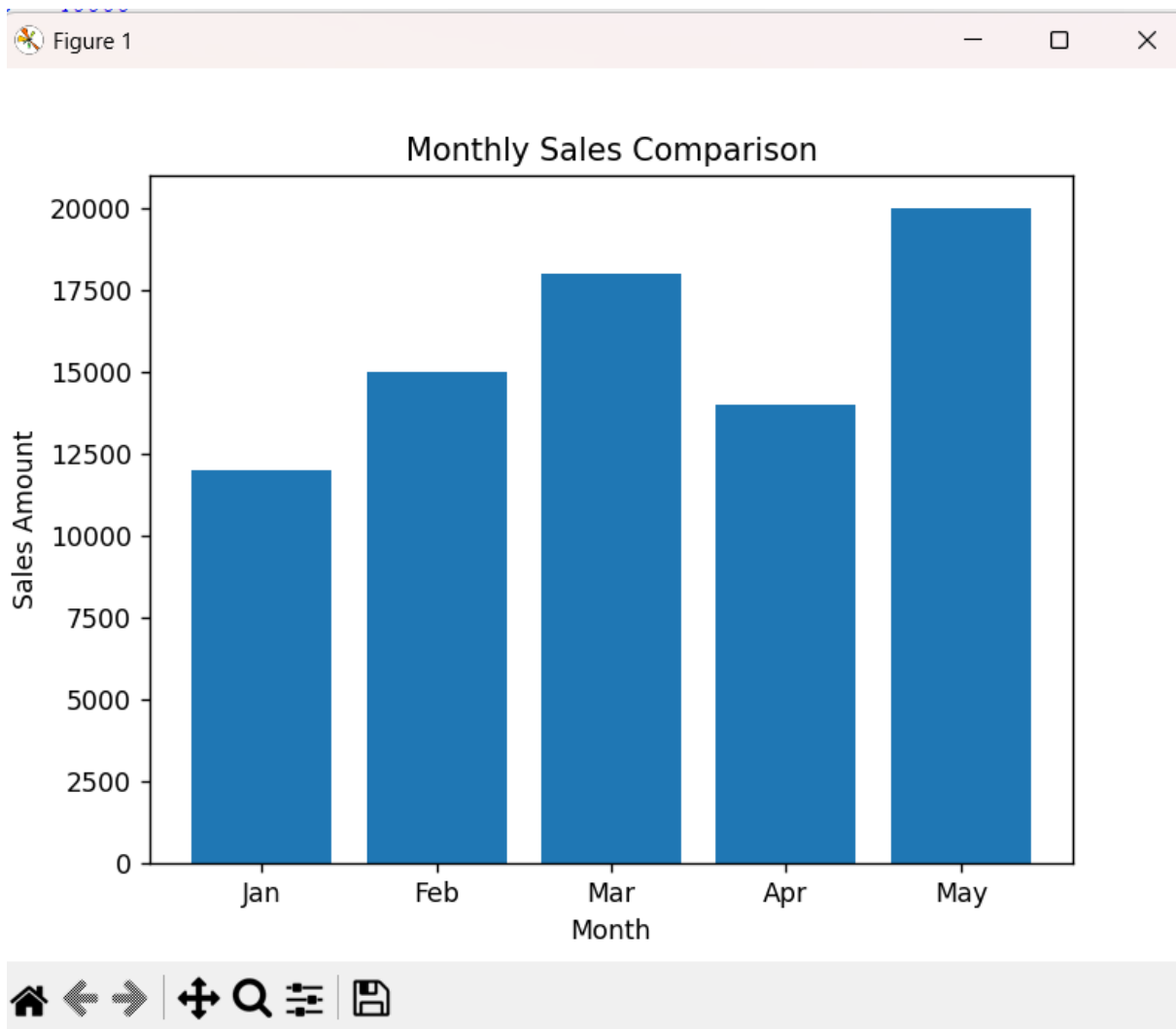
months = ["Jan", "Feb", "Mar", "Apr", "May"]
sales = [12000, 15000, 18000, 14000, 20000]

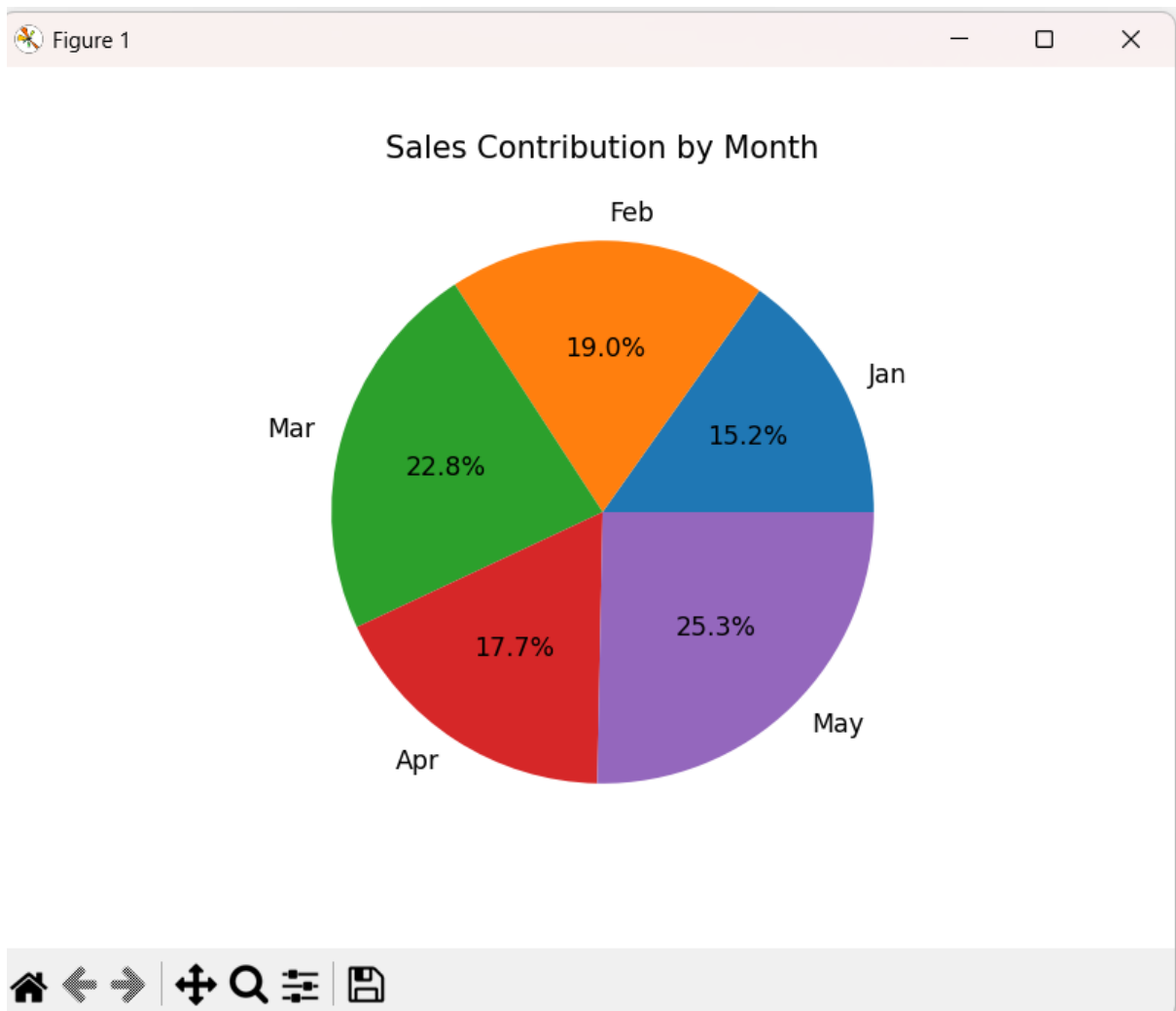
plt.figure()
plt.plot(months, sales)
plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Sales Amount")
plt.show()

plt.figure()
plt.bar(months, sales)
plt.title("Monthly Sales Comparison")
plt.xlabel("Month")
plt.ylabel("Sales Amount")
plt.show()

plt.figure()
plt.pie(sales, labels=months, autopct='%1.1f%%')
plt.title("Sales Contribution by Month")
plt.show()
```







Lab Assignment

Titanic Dataset – Data Analysis & Visualization

Load Dataset

File Edit Format Run Options Window Help

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("titanic.csv")
|

survival = df["Survived"].value_counts()

plt.figure()
plt.bar(survival.index, survival.values)
plt.title("Survival Count")
plt.xlabel("Survived (0 = No, 1 = Yes)")
plt.ylabel("Number of Passengers")
plt.show()

gender_survival = df.groupby("Sex")["Survived"].sum()

plt.figure()
plt.bar(gender_survival.index, gender_survival.values)
plt.title("Survival by Gender")
plt.xlabel("Gender")
plt.ylabel("Survivors")
plt.show()

class_count = df["Pclass"].value_counts()

plt.figure()
plt.bar(class_count.index, class_count.values)
plt.title("Passenger Class Distribution")
plt.xlabel("Class")
plt.ylabel("Count")
plt.show()

plt.figure()
plt.hist(df["Age"].dropna(), bins=10)
plt.title("Age Distribution of Passengers")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```

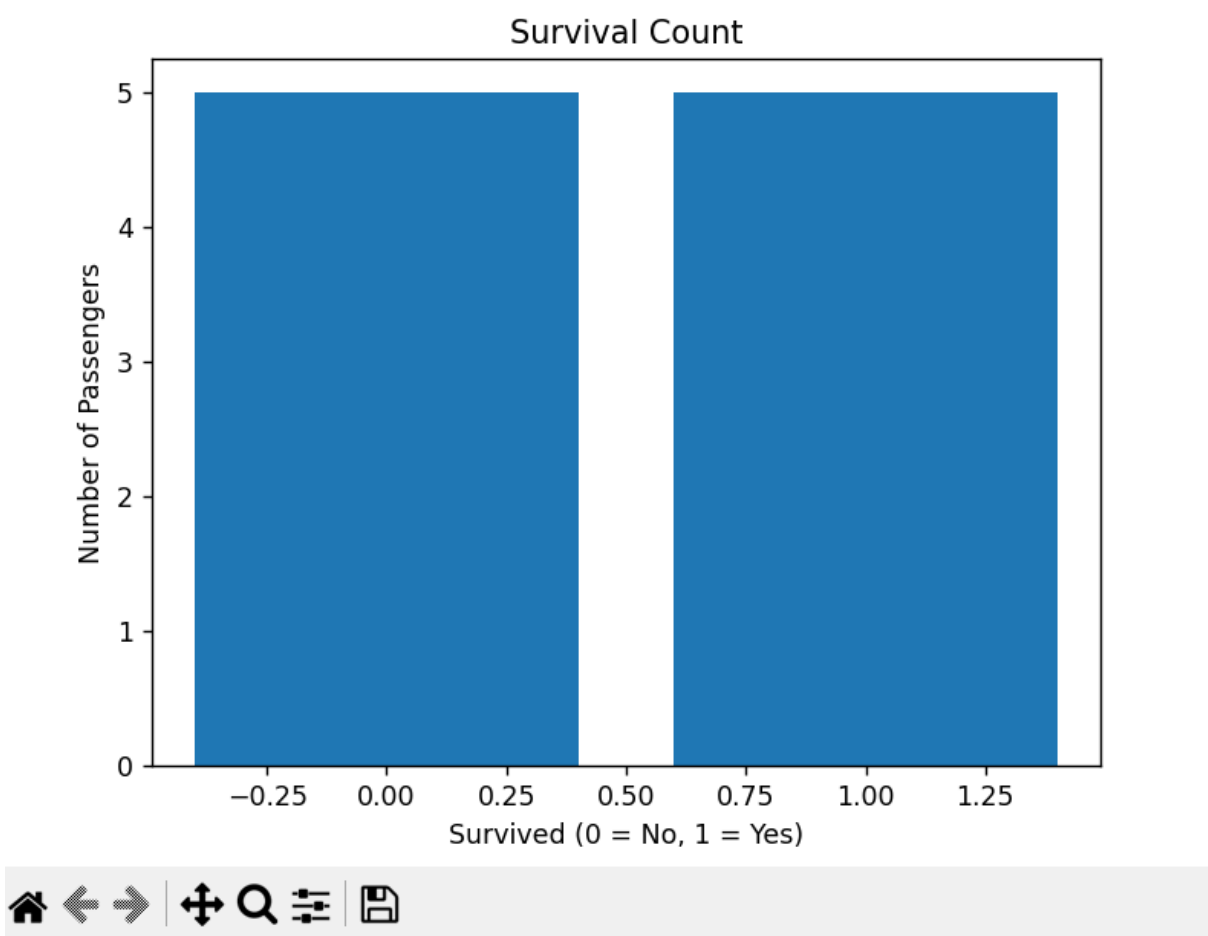


Figure 1

